
Git Workflow & Release Management Guide

GitLab • Branching Strategy • Merge Conflicts • Pull Requests
Git Flow • Conventional Commits • Semantic Versioning • Release Management

A structured reference covering source control workflows and release practices — how teams collaborate on code, manage branches, resolve conflicts, review changes, and ship versioned releases in a disciplined, repeatable way.

Table of Contents

- 83. GitLab
- 84. Branching Strategy
- 85. Merge Conflicts
- 86. Pull Requests
- 87. Git Flow
- 88. Conventional Commits
- 89. Semantic Versioning
- 90. Release Management

GitLab

GitLab is a web-based DevOps platform built around Git repository management, offering source control alongside CI/CD pipelines, issue tracking, code review, container registries, and security scanning in a single integrated tool. It's available as a hosted SaaS product (GitLab.com) and as self-managed software.

Core Features

- **Repository hosting** — Git repos with branching, tags, and web-based file browsing
- **Merge Requests (MRs)** — GitLab's term for pull requests, with inline review, approvals, and discussions
- **CI/CD pipelines** — defined in a `.gitlab-ci.yml` file, running automated build/test/deploy stages
- **Issue tracking & boards** — kanban-style boards, milestones, epics for planning work
- **Container & package registry** — built-in Docker registry and package hosting
- **Security & compliance scanning** — SAST, dependency scanning, and license compliance integrated into pipelines

Example: `.gitlab-ci.yml`

```
stages:
  - test
  - build
  - deploy

run_tests:
  stage: test
  script:
    - pip install -r requirements.txt
    - pytest

deploy_prod:
  stage: deploy
  script:
    - ./deploy.sh
  only:
    - main
```

GitLab vs. GitHub

Both are Git hosting platforms with pull/merge requests and CI/CD, but GitLab ships CI/CD, security scanning, and issue tracking as one integrated product from day one, whereas GitHub composes similar capabilities (Actions, Advanced Security) as separately evolving features. Team preference often comes down to existing ecosystem, self-hosting needs, and DevOps tooling depth required.

Branching Strategy

A branching strategy is a team's agreed-upon convention for how branches are created, named, merged, and released — ensuring parallel work doesn't collide and that the codebase stays deployable.

Common Strategies

Strategy	Description
Trunk-based development	Everyone commits small, frequent changes directly (or via short-lived branches) to one main branch
Git Flow	Long-lived develop/main branches plus feature, release, and hotfix branches (see Topic 87)
GitHub Flow	Simple model: main is always deployable; feature branches merge via PR after review
GitLab Flow	Adds environment branches (e.g. staging, production) on top of GitHub Flow for deployment tracking

Common Branch Naming Conventions

- feature/short-description — new functionality
- bugfix/short-description — non-urgent bug fixes
- hotfix/short-description — urgent production fixes
- release/1.4.0 — release preparation branches
- chore/short-description — tooling, dependency, or maintenance work

Choosing a Strategy

- **Trunk-based / GitHub Flow** — best for teams with strong CI/CD and frequent deploys
- **Git Flow** — best for products with scheduled releases and multiple supported versions
- **GitLab Flow** — a middle ground when explicit environment tracking matters

Key principle: *Whatever the strategy, keep branches short-lived where possible — long-lived branches accumulate drift from main and increase the risk and size of merge conflicts.*

Merge Conflicts

A merge conflict occurs when Git can't automatically reconcile changes from two branches because they modified the same lines (or one branch deleted a file the other modified). Git pauses the merge and asks a human to resolve the conflict.

What a Conflict Looks Like

```
<<<<<<< HEAD
def calculate_total(items):
    return sum(item.price for item in items)
=====
def calculate_total(items):
    return sum(item.price * item.qty for item in items)
>>>>>>> feature/quantity-support
```

Resolution Steps

- 1. Run `git merge` or `git rebase`; Git marks conflicting files
- 2. Open each conflicted file and look for `<<<<<<< / ===== / >>>>>>>` markers
- 3. Decide which changes to keep — yours, theirs, or a manual combination
- 4. Remove the conflict markers and save the resolved file
- 5. Stage the resolved files with `git add`
- 6. Complete the merge with `git commit` (or `git rebase --continue`)
- 7. Re-run tests before pushing to confirm the resolution didn't break anything

Reducing Merge Conflicts

- Keep branches short-lived and merge/rebase from main frequently
- Break large features into smaller, incremental PRs
- Communicate with teammates when touching shared files (config, schema, shared utilities)
- Use consistent code formatting to avoid noisy whitespace-only diffs
- Prefer smaller, focused commits over large sweeping changes

Pull Requests

A pull request (PR) — called a merge request on GitLab — is a proposal to merge changes from one branch into another, providing a structured space for code review, automated checks, and discussion before changes land in the target branch.

Typical PR Lifecycle

- **1. Open** — author pushes a branch and opens a PR against main/develop with a description of the change
- **2. Automated checks** — CI runs tests, linting, and builds; status checks must pass
- **3. Review** — teammates leave inline comments, request changes, or approve
- **4. Iterate** — author pushes additional commits addressing feedback
- **5. Approve** — required number of approvals is reached
- **6. Merge** — changes are merged (merge commit, squash, or rebase) into the target branch
- **7. Cleanup** — the feature branch is typically deleted after merge

Merge Strategies

Strategy	Effect on History
Merge commit	Preserves full branch history plus an explicit merge commit
Squash and merge	Combines all commits into a single commit on the target branch — clean, linear history
Rebase and merge	Replays commits individually onto the target branch — linear history without a merge commit

Good PR Practices

- Keep PRs small and focused on a single change — easier and faster to review
- Write a clear description: what changed, why, and how to test it
- Link related issues/tickets
- Respond to review comments promptly and avoid force-pushing over review history mid-discussion
- Require passing CI checks before merge is allowed

Git Flow

Git Flow is a branching model (introduced by Vincent Driessen) that defines a strict set of long-lived and supporting branches, designed for projects with scheduled release cycles and the need to maintain multiple production versions.

Branch Types

Branch	Purpose
main / master	Always reflects production-ready code; tagged with release versions
develop	Integration branch where completed features accumulate before a release
feature/*	Branched from develop for new features; merged back into develop
release/*	Branched from develop to prepare a release (version bump, final fixes); merged into main and develop
hotfix/*	Branched from main to urgently patch production; merged into main and develop

Typical Flow

```
git checkout -b feature/user-login develop
# ... work, commit ...
git checkout develop
git merge --no-ff feature/user-login

git checkout -b release/1.2.0 develop
# ... final testing, version bump ...
git checkout main
git merge --no-ff release/1.2.0
git tag v1.2.0
git checkout develop
git merge --no-ff release/1.2.0
```

Trade-off: Git Flow provides strong structure for scheduled, versioned releases (e.g. desktop software, libraries with LTS versions) but its long-lived branches and multiple merge steps add overhead for teams doing continuous deployment — many such teams prefer GitHub Flow or trunk-based development instead.

Conventional Commits

Conventional Commits is a lightweight specification for structuring commit messages so they are both human-readable and machine-parseable — enabling automated changelog generation and semantic version bumps directly from commit history.

Format

```
<type>[optional scope]: <description>
```

```
[optional body]
```

```
[optional footer(s)]
```

Common Types

Type	Meaning	Version Impact
feat	A new feature	MINOR bump
fix	A bug fix	PATCH bump
docs	Documentation only changes	None
style	Formatting, no code logic change	None
refactor	Code change that neither fixes a bug nor adds a feature	None
test	Adding or correcting tests	None
chore	Build process or tooling changes	None
feat! / BREAKING CHANGE:	A breaking API change	MAJOR bump

Examples

```
feat(auth): add OAuth2 login support
```

```
fix(api): correct pagination off-by-one error
```

```
feat(api)!: remove deprecated /v1/users endpoint
```

```
BREAKING CHANGE: /v1/users is removed, use /v2/users instead
```

Why it matters: Tools like *semantic-release* can parse conventional commits to automatically determine the next version number and generate a changelog, removing manual guesswork from the release process (see Topics 89 and 90).

Semantic Versioning

Semantic Versioning (SemVer) is a versioning convention using a three-part number, MAJOR.MINOR.PATCH, where each segment's increment communicates the nature and risk of the change to consumers of the software.

Version Format: MAJOR.MINOR.PATCH

Segment	Incremented When	Example
MAJOR	Incompatible / breaking API changes	1.4.2 → 2.0.0
MINOR	New functionality added in a backward-compatible way	1.4.2 → 1.5.0
PATCH	Backward-compatible bug fixes	1.4.2 → 1.4.3

Additional Labels

- **Pre-release** — appended with a hyphen, e.g. 1.5.0-beta.1, indicates an unstable version
- **Build metadata** — appended with a plus sign, e.g. 1.5.0+build.42, doesn't affect version precedence
- **0.y.z** — conventionally signals the API is still unstable / in initial development

Why It Matters

- Consumers can safely upgrade PATCH and MINOR versions without expecting breakage
- Dependency managers (npm, pip, Cargo) use SemVer ranges (e.g. ^1.4.0) to automatically allow safe updates
- A MAJOR bump is a clear, explicit signal that consumers must review changes before upgrading

Caveat: *SemVer is a convention, not enforced by tooling — it only provides real value if maintainers are disciplined about correctly classifying each change.*

Release Management

Release management is the process of planning, scheduling, building, testing, and deploying software releases in a controlled, repeatable, and auditable way — tying together branching strategy, versioning, and deployment into one workflow.

Core Activities

- **Release planning** — deciding what features/fixes go into a given release and its target date
- **Version bumping** — applying SemVer rules to determine the next version number
- **Changelog generation** — documenting what changed, often auto-generated from Conventional Commits
- **Build & artifact creation** — compiling, packaging, and publishing release artifacts (containers, packages, binaries)
- **Testing/staging** — running the release through QA or staging environments before production
- **Deployment** — rolling the release out to production, often gradually
- **Tagging** — marking the exact commit as a release in version control (e.g. `git tag v2.1.0`)
- **Rollback plan** — a clear, tested way to revert if the release causes problems

Common Release Strategies

Strategy	Description
Blue-green deployment	Two identical environments; traffic switches instantly from old to new
Canary release	New version is rolled out to a small subset of traffic before full rollout
Rolling deployment	Instances are updated gradually, a few at a time, with no downtime
Feature-flagged release	Code is deployed dark and enabled later via a feature flag (see Topic 148)

Automating Releases

```
# Example: semantic-release automatically determines next version
# from Conventional Commits and publishes a release
npx semantic-release
# -> analyzes commits since last release
# -> determines version bump (major/minor/patch)
# -> generates changelog
# -> tags the release and publishes artifacts
```

Why it matters: Mature release management reduces the risk and manual toil of shipping software, turning releases from stressful, error-prone events into routine, low-drama occurrences.